

Gestione Prenotazioni di Visite in uno Studio Medico

Progetto back-end in Spring Boot — Specifica Progettuale

Spring Boot 3.x · Java 17 · PostgreSQL · JPA/Hibernate · Spring Security · Swagger/OpenAPI

Versione 1.0 · Aprile 2026

1. Obiettivo

Il progetto consiste nella realizzazione di una piccola applicazione back-end in Spring Boot per la gestione delle prenotazioni di visite mediche. Il sistema deve esporre una REST API completa che permetta di inserire, visualizzare, aggiornare ed eliminare prenotazioni, con supporto alla ricerca filtrata dei dati.

L'applicazione deve essere strutturata in layer separati (controller, service, repository, entity), persistere i dati su PostgreSQL tramite JPA/Hibernate e includere validazione dei campi, gestione centralizzata degli errori, sicurezza di base con ruoli differenti (user e admin), documentazione Swagger/OpenAPI, endpoint informativi e dati di esempio caricati all'avvio.

2. Stack tecnologico

Componente	Tecnologia
Framework	Spring Boot 3.x
Linguaggio	Java 17+
Persistenza	Spring Data JPA / Hibernate
Database	PostgreSQL
Validazione	Jakarta Bean Validation
Sicurezza	Spring Security
Documentazione API	Springdoc OpenAPI / Swagger UI
Build tool	Maven

3. Struttura del progetto

Il progetto deve seguire una struttura a package ben definita che rispetti la separazione delle responsabilità:

```
src/main/java/com/studio-medico/  
■■■■ controller/    PatientController, DoctorController, AppointmentController  
■■■■ service/       PatientService, DoctorService, AppointmentService  
■■■■ repository/    PatientRepository, DoctorRepository, AppointmentRepository  
■■■■ entity/        Patient, Doctor, Appointment  
■■■■ dto/           PatientRequestDTO, PatientResponseDTO,  
■■                DoctorRequestDTO, DoctorResponseDTO,  
■■                AppointmentRequestDTO, AppointmentResponseDTO  
■■■■ enums/         AppointmentStatus, VisitType  
■■■■ exception/     ResourceNotFoundException, GlobalExceptionHandler  
■■■■ config/        SecurityConfig, SwaggerConfig, DataInitializer
```

4. Modello dati

4.1 Enum

Enum	Valori
AppointmentStatus	BOOKED · CONFIRMED · CANCELLED · COMPLETED
VisitType	GENERAL_CHECKUP · CARDIOLOGY · DERMATOLOGY · ORTHOPEDICS

4.2 Patient

Campo	Tipo Java	Vincoli
id	UUID	PK, auto-generato
firstName	String	@NotBlank

Campo	Tipo Java	Vincoli
lastName	String	@NotBlank
email	String	@NotBlank, @Email, unique
phoneNumber	String	opzionale
fiscalCode	String	@NotBlank, unique

4.3 Doctor

Campo	Tipo Java	Vincoli
id	UUID	PK, auto-generato
firstName	String	@NotBlank
lastName	String	@NotBlank
specialization	String	@NotBlank
email	String	@NotBlank, @Email, unique

4.4 Appointment

Campo	Tipo Java	Vincoli
id	UUID	PK, auto-generato
appointmentDate	LocalDate	@NotNull
appointmentTime	LocalTime	@NotNull
visitType	VisitType	@NotNull, @Enumerated(String)
status	AppointmentStatus	@NotNull, @Enumerated(String)
notes	String	opzionale
createdAt	LocalDateTime	@CreationTimestamp, auto-impostato
patient	Patient	FK patients, @ManyToOne
doctor	Doctor	FK doctors, @ManyToOne

4.5 Relazioni

Relazione	Tipo JPA	Descrizione
Patient Appointment	OneToMany	Un paziente può avere molte prenotazioni
Doctor Appointment	OneToMany	Un medico può avere molte prenotazioni
Appointment Patient	ManyToOne	Una prenotazione appartiene a un solo paziente
Appointment Doctor	ManyToOne	Una prenotazione è associata a un solo medico

5. Endpoint REST

5.1 Patient — /api/patients

Metodo	URL	Descrizione
POST	/api/patients	Crea un nuovo paziente.
GET	/api/patients	Elenco pazienti con filtri opzionali.
GET	/api/patients/{id}	Dettaglio paziente per ID.
PUT	/api/patients/{id}	Aggiornamento completo del paziente.

Metodo	URL	Descrizione
DELETE	/api/patients/{id}	Elimina il paziente.
GET	/api/patients/{id}/appointments	Tutte le prenotazioni del paziente.

Filtri query string: firstName, lastName, email, fiscalCode. Se non vengono passati parametri, vengono restituiti tutti i pazienti; se valorizzati, l'elenco viene filtrato combinando i criteri.

5.2 Doctor — /api/doctors

Metodo	URL	Descrizione
POST	/api/doctors	Crea un nuovo medico.
GET	/api/doctors	Elenco medici con filtri opzionali.
GET	/api/doctors/{id}	Dettaglio medico per ID.
PUT	/api/doctors/{id}	Aggiornamento completo del medico.
DELETE	/api/doctors/{id}	Elimina il medico.
GET	/api/doctors/{id}/appointments	Tutte le prenotazioni del medico.

Filtri query string: firstName, lastName, specialization, email.

5.3 Appointment — /api/appointments

Metodo	URL	Descrizione
POST	/api/appointments	Crea una nuova prenotazione associandola a paziente e medico esistenti.
GET	/api/appointments	Elenco prenotazioni con filtri opzionali, combinabili tra loro.
GET	/api/appointments/{id}	Dettaglio prenotazione per ID.
PUT	/api/appointments/{id}	Aggiornamento completo della prenotazione.
DELETE	/api/appointments/{id}	Elimina la prenotazione.

Filtri query string: appointmentDate, status, visitType, doctorId, patientId. I filtri possono essere combinati liberamente.

Campi aggiornabili tramite PUT: appointmentDate, appointmentTime, visitType, status, notes, patientId, doctorId.

Tutte le eccezioni devono essere intercettate da un unico handler annotato con `@RestControllerAdvice`. Le risposte di errore devono essere in formato JSON e contenere almeno timestamp, codice HTTP, messaggio descrittivo e path della richiesta.

Scenario	Eccezione	HTTP
Risorsa non trovata	ResourceNotFoundException	404 Not Found
Dati non validi (validazione)	MethodArgumentNotValidException	400 Bad Request
Duplicato su campo unique	DataIntegrityViolationException	409 Conflict
Errore generico non gestito	Exception	500 Internal Server Error

Esempio di risposta di errore:

```
{
  "timestamp": "2026-04-20T16:00:00",
  "status": 404,
  "error": "Not Found",
  "message": "Paziente con id 5 non trovato",
  "path": "/api/patients/5"
}
```

6. Sicurezza con Spring Security (facoltativo, se ce la fai con il tempo)

L'applicazione deve prevedere almeno due ruoli distinti, configurati tramite HTTP Basic Authentication con utenti in memoria. La configurazione deve differenziare chiaramente le operazioni consentite a ciascun ruolo.

Ruolo	Operazioni consentite
USER	Sola lettura — accesso a tutti gli endpoint GET.
ADMIN	Lettura e scrittura — accesso a tutti i metodi (GET, POST, PUT, DELETE).

- Gli endpoint di Swagger UI, OpenAPI JSON e Actuator devono essere accessibili senza autenticazione.
- E' consigliato l'uso di `BcryptPasswordEncoder` per salvare le password.

7. Documentazione Swagger / OpenAPI

La documentazione deve essere accessibile tramite Swagger UI e deve esporre tutti gli endpoint con i dettagli di input/output. È sufficiente configurare titolo, versione e descrizione dell'API tramite la classe di configurazione OpenAPI.

- Swagger UI: `http://localhost:8080/swagger-ui/index.html`
- OpenAPI JSON: `http://localhost:8080/v3/api-docs`

8. Dati di esempio all'avvio

All'avvio deve essere eseguito un `CommandLineRunner` o `ApplicationRunner` che carichi dati iniziali, solo se il database è vuoto.

Tipo	Dati di esempio
Doctor	Mario Rossi – Cardiologia; Anna Bianchi – Dermatologia
Patient	Luca Verdi (CF: VRDLU90A01H501Z); Giulia Neri (CF: NRIGLL95B41F205X)
Appointment	Almeno due prenotazioni con date, orari, visitType e status differenti

Dipendenze Maven principali

- `spring-boot-starter-web`
- `spring-boot-starter-data-jpa`
- `spring-boot-starter-security`
- `spring-boot-starter-validation`
- `postgresql` (scope runtime)
- `springdoc-openapi-starter-webmvc-ui`
- `lombok` (opzionale)

9. Note implementative

- Usare DTO separati per request e response, evitando di esporre direttamente le entity JPA.
- I service devono contenere la logica applicativa; i controller devono restare snelli.
- L'aggiornamento tramite PUT deve gestire esplicitamente tutti i campi, incluse le associazioni.

Le relazioni in cascata vanno gestite con attenzione per evitare eliminazioni indesiderate.

10. Checklist dei requisiti

Requisito	
CRUD completo per Patient, Doctor e Appointment	✓
Ricerca filtrata con parametri query opzionali nelle getAll	✓
Endpoint aggiuntivi: prenotazioni per medico e per paziente	✓
Persistenza su PostgreSQL tramite JPA/Hibernate	✓
Validazione dei campi obbligatori con @Valid	✓
Gestione centralizzata delle eccezioni con @RestControllerAdvice	✓
Risposte di errore in formato JSON con codice HTTP corretto	✓
Documentazione Swagger / OpenAPI	✓
Dati di esempio caricati automaticamente all'avvio	✓
Sicurezza con Spring Security e due ruoli distinti (USER / ADMIN) (facoltativo)	✓